

Taller practico de MongoDB

Kotlin y KMongo

Guia progresiva para trabajar con MongoDB Atlas desde Kotlin

Documento generado a partir de doc/taller_mongodb.md

Proyecto 2526_PRO_U8_mongo

8 de mayo de 2026

Índice

Taller práctico de MongoDB con Kotlin y KMongo	3
1. Objetivo	3
2. Requisitos	3
3. Introducción a MongoDB	3
3.1. Qué es MongoDB	3
3.2. Cómo se trabaja con MongoDB	4
3.3. Por qué vamos a usarlo así en este taller	4
4. MongoDB y KMongo en este taller	4
4.1. Idea general	4
4.2. Clases principales del driver oficial	5
4.3. Clases BSON que también usamos	5
4.4. Utilidades de KMongo que vamos a utilizar	6
5. Diagramas de apoyo	6
5.1. Diagrama de clases	6
5.2. Diagrama de secuencia	8
5.3. Dominio del taller y lectura de los diagramas	10
6. Módulo 0. Configuración del entorno	12
6.1. Qué vas a aprender	12
6.2. Variables de entorno	12
6.3. Configuración en Kotlin	12
6.4. Conexión a MongoDB	12
7. Módulo 1. Gestión de bases de datos	13
7.1. Qué vas a aprender	13
7.2. Listar bases de datos	13
7.3. Seleccionar una base de datos	13
7.4. Materializar una base de datos	13
7.5. Eliminar una base de datos	13
7.6. Ejercicio 1. Explorar bases de datos	13
8. Módulo 2. Gestión de colecciones	14
8.1. Qué vas a aprender	14
8.2. Listar colecciones	14
8.3. Crear una colección explícitamente	14
8.4. Crear una colección de forma implícita	15
8.5. Renombrar y eliminar colecciones	15
8.6. Ejercicio 2. Colecciones de una biblioteca	15
9. Módulo 3. Operaciones CRUD sobre documentos	15
9.1. Qué vas a aprender	16
9.2. Crear el servicio de productos	16
9.3. Insertar documentos	16
9.4. Consultar documentos	17
9.5. Actualizar documentos	17
9.6. Eliminar documentos	17
9.7. Ejercicio 3. Catálogo de productos	18
9.8. Ejercicio 4. Actualización de inventario	18
10. Módulo 4. Proyecto integrado de biblioteca digital	19

10.1. Qué vas a aprender	19
10.2. Modelos principales	19
10.3. Crear el servicio de biblioteca	20
10.4. Operaciones disponibles	20
10.5. Ejercicio 5. Biblioteca digital completa	21
11. Pruebas del proyecto	22
11.1. Qué se prueba	22
11.2. Herramientas usadas	22
12. Ejecutar el proyecto	22
12.1. Ejecutar las pruebas	22
12.2. Ejecutar la aplicación	23
13. Buenas prácticas aplicadas	23

Taller práctico de MongoDB con Kotlin y KMongo

1. Objetivo

En este taller aprenderás a trabajar con MongoDB Atlas desde una aplicación Kotlin. El recorrido está organizado de forma progresiva:

1. Configurar la conexión a MongoDB sin escribir credenciales en el código.
2. Gestionar bases de datos.
3. Gestionar colecciones.
4. Realizar operaciones CRUD sobre documentos.
5. Construir un pequeño sistema integrado de biblioteca digital.

La implementación de referencia está en:

`src/main/kotlin/org/iesra/tallermongo/`

Las pruebas unitarias están en:

`src/test/kotlin/org/iesra/tallermongo/`

2. Requisitos

- JDK 21.
- Gradle con Kotlin DSL.
- Cuenta en MongoDB Atlas.
- Cluster de MongoDB Atlas creado.
- Usuario de base de datos configurado en Atlas.
- IP autorizada en el panel de acceso de Atlas.

En este proyecto usarás Kotlin y KMongo:

```
implementation("org.litote.kmongo:kmongo:4.11.0")
```

Para las pruebas unitarias usarás Kotest con DescribeSpec y MockK.

3. Introducción a MongoDB

3.1. Qué es MongoDB

MongoDB es una base de datos NoSQL orientada a documentos. En lugar de organizar la información en filas y columnas, guarda los datos en documentos con una estructura parecida a JSON.

Esto hace que encaje muy bien con la forma en la que programamos en Kotlin, porque los datos de la base de datos se parecen mucho a los objetos del código.

Los tres conceptos básicos que debes recordar son:

- un **documento** es la unidad básica de información
- una **colección** agrupa documentos relacionados
- una **base de datos** agrupa colecciones

Ejemplo de documento:

```
{
  "_id": "...",
  "nombre": "Libro Kotlin",
  "precio": 34.95,
  "stock": 20,
  "categoria": "libros"
}
```

3.2. Cómo se trabaja con MongoDB

Trabajar con MongoDB suele implicar estos pasos:

1. Conectarse al servidor o al clúster.
2. Seleccionar una base de datos.
3. Crear o usar colecciones.
4. Insertar, consultar, actualizar y eliminar documentos.
5. Diseñar un modelo de datos que encaje con la aplicación.

MongoDB almacena internamente la información en BSON, que es una representación binaria de JSON. Como desarrolladores, normalmente trabajamos con documentos, campos y colecciones, mientras que el driver se encarga de la traducción entre nuestras clases Kotlin y ese formato interno.

3.3. Por qué vamos a usarlo así en este taller

MongoDB te resultará útil cuando quieras:

- trabajar con datos que evolucionan con facilidad
- representar información compleja sin un esquema rígido
- acercar el modelo de datos al modelo de objetos del programa
- construir aplicaciones modernas con una base de datos flexible

En este taller no cubrirás toda la plataforma MongoDB. Te vas a centrar en lo que más te ayuda a aprender programación con bases de datos:

- conexión
- bases de datos
- colecciones
- documentos
- operaciones CRUD
- una estructura sencilla de clases en Kotlin para trabajar con MongoDB de forma ordenada

4. MongoDB y KMongo en este taller

4.1. Idea general

En el proyecto intervienen tres niveles:

1. Nuestras clases del taller, que organizan el código.
2. KMongo, que ofrece una API más cómoda para Kotlin.

3. El driver oficial de MongoDB y las clases BSON, que son la base real de la conexión y de las operaciones.

La idea es que no empieces directamente con una API grande y dispersa. Primero verás una capa pequeña y comprensible, y luego irás conectando esa capa con las clases reales que ofrece MongoDB.

4.2. Clases principales del driver oficial

Las clases más importantes para trabajar con MongoDB en Kotlin son estas:

Clase	Qué representa	Uso en el taller
KMongo	Punto de entrada de KMongo	Crear el cliente usando la URI de conexión
MongoClient	Cliente de conexión	Abrir la conexión al clúster y acceder a bases de datos
MongoDatabase	Base de datos concreta	Gestionar colecciones dentro de una base de datos
MongoCollection	Colección tipada	Trabajar con documentos de un tipo concreto

Imports habituales:

```
import com.mongodb.client.MongoClient
import com.mongodb.client.MongoCollection
import com.mongodb.client.MongoDatabase
import org.litote.kmongo.KMongo
```

Jerarquía básica:

```
val client: MongoClient = KMongo.createClient(config.uri)
val database: MongoDatabase = client.getDatabase("taller_mongo")
val products: MongoCollection<Product> = database.getCollection("productos")
```

4.3. Clases BSON que también usamos

Clase	Qué representa	Uso en el taller
Document	Un documento genérico sin tipar	Crear documentos de prueba y algunos comandos sencillos
ObjectId	Identificador habitual de MongoDB	Generar el valor por defecto de <code>_id</code> en los modelos

Imports habituales:

```
import org.bson.Document
import org.bson.types.ObjectId
```

Ejemplo con ObjectId:

```
data class Product(
    val _id: String = ObjectId().toHexString(),
```

```

    val nombre: String,
    val precio: Double,
    val stock: Int,
    val categoria: String,
)

```

Ejemplo con Document:

```
database.getCollection(collectionName).insertOne(Document("creada", true))
```

4.4. Utilidades de KMongo que vamos a utilizar

Utilidad	Qué hace	Uso en el taller
KMongo	Crea el cliente MongoDB	Se usa en MongoClient
getCollection	Obtiene colecciones tipadas	Pedir colecciones de Product o Libro
eq	Filtro de igualdad	Buscar por nombre, título o identificador
gt	Filtro "mayor que"	Buscar productos por precio mínimo
lte	Filtro "menor o igual que"	Detectar productos o libros sin unidades
inc	Incrementa un valor numérico	Sumar stock o copias
setValue	Cambia el valor de un campo	Actualizar precio, descuento o disponibilidad
unset	Elimina un campo	Quitar el campo descuento
ascending	Orden ascendente	Ordenar libros por título
replaceOne	Sustituye un documento con un filtro	Implementar upsert
WithFilter	Inserta si no existe o reemplaza	Completar la operación de upsert

En código, la utilidad de reemplazo se escribe sin espacio: `replaceOneWithFilter`.

Por ejemplo, esta consulta:

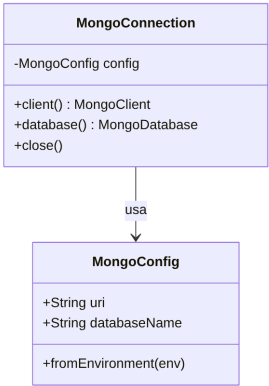
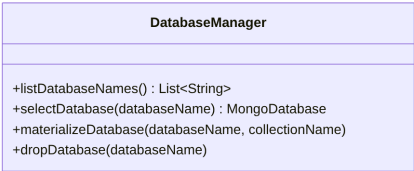
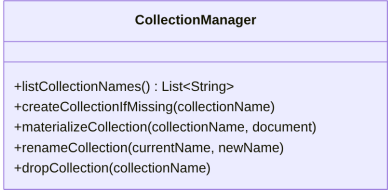
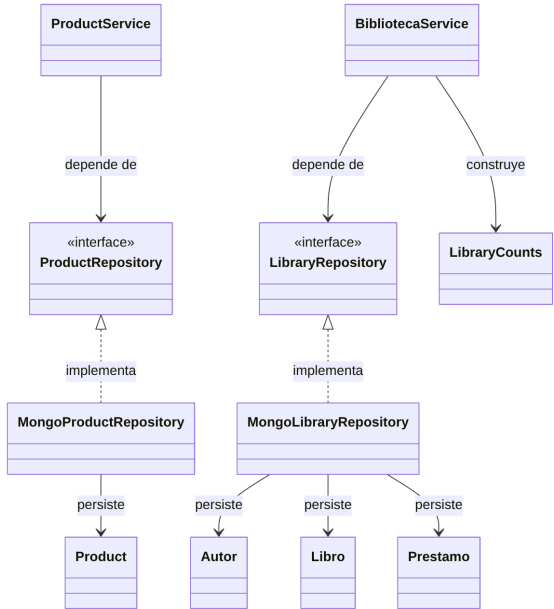
```
collection.find(Product::precio gt minimumPrice).toList()
```

te resultará más legible que construir a mano un documento BSON con operadores de bajo nivel.

5. Diagramas de apoyo

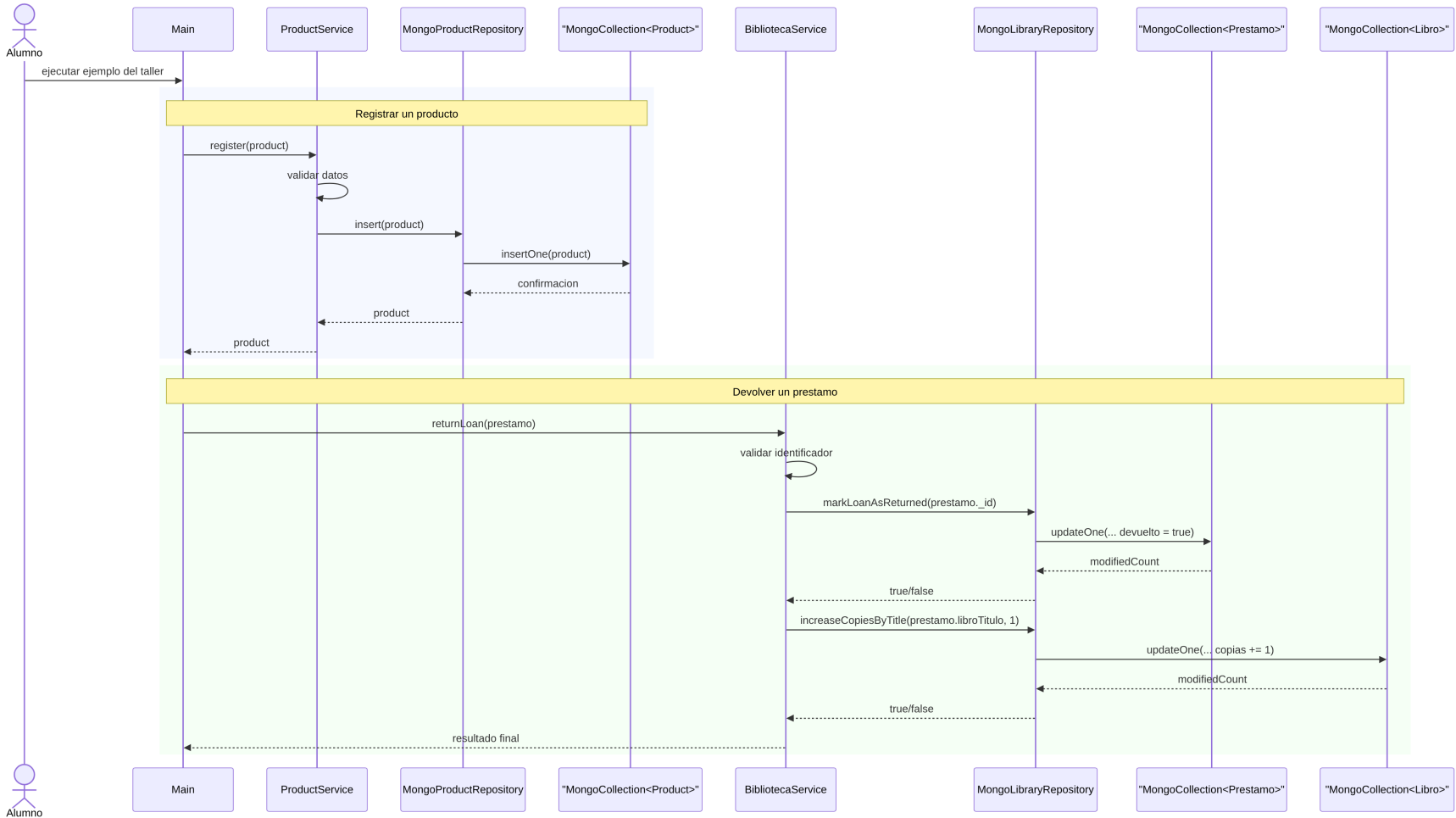
5.1. Diagrama de clases

Este diagrama presenta la estructura general que iremos construyendo a lo largo del taller.



5.2. Diagrama de secuencia

Este diagrama resume dos operaciones básicas del taller: registrar un producto y devolver un préstamo.



5.3. Dominio del taller y lectura de los diagramas

Antes de entrar en los módulos prácticos, te conviene entender qué problema modela el taller y por qué aparecen estas clases.

En el taller trabajarás sobre dos dominios sencillos pero muy útiles para aprender MongoDB:

1. Un **catálogo de productos**, que te permite practicar operaciones CRUD sobre una única colección.
2. Una **biblioteca digital**, que te permite trabajar con varias colecciones relacionadas: autores, libros y préstamos.

Sobre esos dos dominios construiremos una pequeña arquitectura en Kotlin. La idea no es esconder MongoDB, sino organizar su uso para que entiendas mejor qué responsabilidad tiene cada pieza.

5.3.1. Qué representa cada clase y cómo se relaciona con las demás

- **MongoConfig**: concentra la configuración de conexión. Sirve para leer la URI y el nombre de la base de datos desde variables de entorno y validar que esos valores existen.
- **MongoConnection**: encapsula la apertura y el cierre del cliente MongoDB. Se apoya en **MongoConfig** y ofrece acceso al **MongoClient** y a la **MongoDatabase** configurada.
- **DatabaseManager**: agrupa las operaciones básicas sobre bases de datos. Se usa en la primera parte del taller para aprender a listar, seleccionar, materializar y eliminar bases de datos.
- **CollectionManager**: agrupa las operaciones sobre colecciones. Se apoya en una **MongoDatabase** concreta y te permite listar, crear, renombrar y eliminar colecciones.
- **Product**: representa un documento de la colección productos. Es el modelo del dominio usado en el módulo CRUD.
- **ProductRepository**: define el contrato de persistencia para productos. Su función es separar la lógica de negocio del acceso concreto a MongoDB.
- **MongoProductRepository**: implementa **ProductRepository** usando **MongoCollection<Product>** y utilidades de **KMongo**. Es la clase que realmente ejecuta las operaciones sobre la colección productos.
- **ProductService**: coordina la validación y el uso del repositorio de productos. Es el punto desde el que se lanzan las operaciones del módulo de productos.
- **Autor**: representa un documento de la colección autores.
- **Libro**: representa un documento de la colección libros.
- **Prestamo**: representa un documento de la colección préstamos.
- **LibraryRepository**: define el contrato de persistencia para el dominio de biblioteca. Reúne operaciones que afectan a autores, libros y préstamos.
- **MongoLibraryRepository**: implementa **LibraryRepository** apoyándose en tres colecciones tipadas: autores, libros y préstamos.
- **BibliotecaService**: coordina las validaciones y las operaciones del dominio de biblioteca. Aquí se ve mejor cómo una operación funcional puede implicar varias colecciones.
- **LibraryCounts**: actúa como objeto resumen para devolver el recuento de documentos de las tres colecciones del dominio de biblioteca.

Las relaciones más importantes del diagrama de clases son estas:

- `MongoConnection` usa `MongoConfig` para crear la conexión.
- `ProductService` depende de `ProductRepository`, y `MongoProductRepository` es su implementación real para MongoDB.
- `BibliotecaService` depende de `LibraryRepository`, y `MongoLibraryRepository` es su implementación real para MongoDB.
- Los repositorios MongoDB persisten los modelos del dominio (`Product`, `Autor`, `Libro`, `Prestamo`).
- `BibliotecaService` construye `LibraryCounts` cuando necesita devolver un resumen del estado del sistema.

Esta organización te permite trabajar con MongoDB sin mezclarlo todo en `Main.kt`. Así puedes distinguir mejor entre:

- configuración
- conexión
- acceso a datos
- validación y reglas de negocio
- modelos del dominio

5.3.2. Cómo interactúan las clases en los ejemplos del diagrama de secuencia El diagrama de secuencia muestra dos ejemplos pequeños pero representativos del taller.

Primer ejemplo: registrar un producto

1. `Main` crea o recibe un objeto `Product`.
2. `Main` llama a `ProductService.register(product)`.
3. `ProductService` valida los datos del producto antes de persistirlo.
4. Si el producto es válido, el servicio delega en `MongoProductRepository`.
5. `MongoProductRepository` usa la colección `MongoCollection<Product>` para ejecutar `insertOne(product)`.
6. El resultado vuelve al repositorio, después al servicio y finalmente a `Main`.

Este flujo enseña una idea importante: aunque MongoDB permite insertar directamente en la colección, en el taller añadimos un servicio intermedio para dejar claras las validaciones y la responsabilidad de cada capa.

Segundo ejemplo: devolver un préstamo

1. `Main` llama a `BibliotecaService.returnLoan(prestamo)`.
2. `BibliotecaService` valida primero que el préstamo tenga un identificador correcto.
3. Después pide a `MongoLibraryRepository` que marque el préstamo como devuelto en la colección `prestamos`.
4. A continuación, el mismo servicio pide al repositorio que incremente en una unidad las copias del libro relacionado en la colección `libros`.
5. El servicio combina ambos resultados y devuelve un único resultado final a `Main`.

Este segundo flujo es especialmente útil porque te muestra que una operación funcional aparentemente simple puede implicar varias actualizaciones en distintas colecciones.

En otras palabras:

- el modelo de productos te sirve para aprender CRUD sobre una sola colección

- el modelo de biblioteca te sirve para entender operaciones coordinadas sobre varias colecciones
- los diagramas te ayudan a visualizar esa evolución antes de empezar a programar cada módulo

Con esta introducción ya tenemos el contexto necesario para empezar la parte práctica del taller.

6. Módulo 0. Configuración del entorno

6.1. Qué vas a aprender

Antes de trabajar con MongoDB necesitas configurar la aplicación. La URI de conexión contiene credenciales, por eso no debes escribirla directamente en el código fuente.

6.2. Variables de entorno

Crea tus variables de entorno antes de ejecutar la aplicación:

```
MONGODB_URI=mongodb+srv://usuario:password@cluster.mongodb.net/  
MONGODB_DATABASE=taller_mongo
```

El repositorio incluye un fichero `.env.example` con el formato esperado, pero no debes guardar credenciales reales en Git.

6.3. Configuración en Kotlin

La clase `MongoConfig` lee la configuración desde variables de entorno:

```
val config = MongoConfig.fromEnvironment()
```

La clase valida que la URI y el nombre de la base de datos no estén vacíos. Si no defines `MONGODB_DATABASE`, usarás `taller_mongo` como base de datos por defecto.

6.4. Conexión a MongoDB

La clase `MongoConnection` crea un cliente reutilizable con KMongo. Cumpliendo con uno de los principios de la programación orientada a objetos, el cliente se encapsula dentro de la clase `MongoConnection`.

```
// Parte de la clase MongoConnection. Crea un cliente reutilizable.  
private val client: MongoClient by lazy { KMongo.createClient(config.uri) }
```

Utilizamos este mismo cliente para acceder a la base de datos y otras operaciones. El cliente permanece abierto mientras se necesite y se cierra al finalizar.

```
MongoConnection(config).use { connection ->  
    val database = connection.database()  
    println(database.name)  
}
```

Si usas `use`, te aseguras de que el cliente se cierre al terminar el bloque.

Recuerda: crear un cliente por cada operación es una mala práctica. En una aplicación real se reutiliza el cliente mientras la aplicación está activa.

7. Módulo 1. Gestión de bases de datos

7.1. Qué vas a aprender

MongoDB crea las bases de datos de forma implícita. Acceder a una base de datos no la materializa hasta que insertas el primer documento.

La clase principal de este módulo es `DatabaseManager`.

7.2. Listar bases de datos

Con el método `listDatabaseNames()` puedes obtener una lista de las bases de datos disponibles en el servidor. Ten en cuenta que solo aparecerán las bases de datos que tengan al menos una colección con datos.

```
val databaseManager = DatabaseManager(connection.client())
val names = databaseManager.listDatabaseNames()
println(names)
```

7.3. Seleccionar una base de datos

Con el método `selectDatabase()` puedes obtener una instancia de `MongoDatabase` para una base de datos concreta. Sin embargo, esta operación no crea la base de datos en el servidor ni la hace visible hasta que insertas el primer documento.

```
val academia = databaseManager.selectDatabase("academia")
println(academia.name)
```

En este punto `academia` puede no aparecer todavía al listar bases de datos, porque aún no tiene datos.

7.4. Materializar una base de datos

Con el método `materializeDatabase()` puedes forzar la creación de una base de datos insertando un documento temporal en una colección de prueba. Esto hace que MongoDB reconozca la base de datos y la muestre al listar.

```
databaseManager.materializeDatabase(databaseName = "academia", collectionName = "prueba")
```

Esta operación inserta un documento temporal en una colección de prueba. A partir de ahí, MongoDB ya podrá mostrar la base de datos.

7.5. Eliminar una base de datos

Con el método `dropDatabase()` puedes eliminar una base de datos completa. Ten cuidado, esta operación borra todas las colecciones y documentos que contiene la base de datos.

```
databaseManager.dropDatabase("academia")
```

Cuidado: eliminar una base de datos borra todas sus colecciones y documentos.

7.6. Ejercicio 1. Explorar bases de datos

7.6.1. Objetivo Comprobar cómo MongoDB crea una base de datos solo cuando contiene datos.

7.6.2. Pasos

1. Lista las bases de datos disponibles.
2. Selecciona una base de datos llamada academia.
3. Comprueba que puede no aparecer todavía en la lista.
4. Materializa la base de datos insertando un documento temporal en prueba.
5. Vuelve a listar las bases de datos.
6. Elimina academia.

7.6.3. Solución Debes observar que academia solo aparece tras insertar el primer documento.

```
val databaseManager = DatabaseManager(connection.client())

println(databaseManager.listDatabaseNames())

databaseManager.selectDatabase("academia")
println("academia" in databaseManager.listDatabaseNames())

databaseManager.materializeDatabase("academia")
println("academia" in databaseManager.listDatabaseNames())

databaseManager.dropDatabase("academia")
```

8. Módulo 2. Gestión de colecciones

8.1. Qué vas a aprender

Una colección agrupa documentos relacionados. Se parece a una tabla en bases de datos relacionales, pero MongoDB te permite trabajar con documentos flexibles.

La clase principal de este módulo es `CollectionManager`.

8.2. Listar colecciones

Con el método `listCollectionNames()` puedes obtener una lista de las colecciones disponibles en una base de datos concreta. Al igual que con las bases de datos, solo aparecerán las colecciones que tengan al menos un documento.

```
val collectionManager = CollectionManager(database)
println(collectionManager.listCollectionNames())
```

8.3. Crear una colección explícitamente

Con el método `createCollectionIfMissing()` puedes crear una colección de forma explícita. Sin embargo, ten en cuenta que MongoDB no mostrará la colección al listar hasta que insertes el primer documento.

```
collectionManager.createCollectionIfMissing("libros")
```

8.4. Crear una colección de forma implícita

Con el método `materializeCollection()` puedes forzar la creación de una colección insertando un documento temporal. Esto hace que MongoDB reconozca la colección y la muestre al listar.

```
collectionManager.materializeCollection("usuarios")
```

La colección se crea cuando insertas el primer documento.

8.5. Renombrar y eliminar colecciones

Con el método `renameCollection()` puedes cambiar el nombre de una colección. Con `dropCollection()` puedes eliminar una colección completa. Ten cuidado, esta última operación borra todos los documentos que contiene la colección.

```
collectionManager.renameCollection("usuarios", "socios")
collectionManager.dropCollection("socios")
```

8.6. Ejercicio 2. Colecciones de una biblioteca

8.6.1. Objetivo Crear y gestionar colecciones para una biblioteca.

8.6.2. Pasos

1. Selecciona la base de datos biblioteca.
2. Crea las colecciones libros, autores y socios.
3. Lista las colecciones.
4. Renombra socios a usuarios.
5. Elimina autores.
6. Muestra el estado final.

8.6.3. Solución

```
val database = databaseManager.selectDatabase("biblioteca")
val collections = CollectionManager(database)
```

```
collections.createCollectionIfMissing("libros")
collections.createCollectionIfMissing("autores")
collections.createCollectionIfMissing("socios")
```

```
println(collections.listCollectionNames())
```

```
collections.renameCollection("socios", "usuarios")
collections.dropCollection("autores")
```

```
println(collections.listCollectionNames())
```

9. Módulo 3. Operaciones CRUD sobre documentos

Para trabajar con documentos, lo ideal es usar colecciones tipadas. En este módulo usarás `MongoCollection<Product>` para gestionar productos de un catálogo.

Ten en cuenta que MongoDB no impone un esquema rígido, pero al usar colecciones tipadas con KMongo puedes beneficiarte de la seguridad de tipos y de una API más cómoda para construir consultas.

La colección `productos` se usará para practicar las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre documentos de tipo `Product`.

Será el repositorio `MongoProductRepository` el que implemente esas operaciones usando KMongo, y el servicio `ProductService` el que coordine la lógica de negocio y las validaciones.

9.1. Qué vas a aprender

En MongoDB los datos se guardan como documentos BSON. En Kotlin trabajarás con `data class` para representar esos documentos de forma clara y tipada.

El modelo usado en este módulo es `Product`:

```
import org.bson.types.ObjectId

data class Product(
    val _id: String = ObjectId().toHexString(),
    val nombre: String,
    val precio: Double,
    val stock: Int,
    val categoria: String,
    val categorias: List<String> = emptyList(),
    val descuento: Int? = null,
)
```

Las operaciones se organizan en dos piezas:

- `ProductRepository`: contrato de acceso a datos.
- `MongoProductRepository`: implementación con KMongo.
- `ProductService`: validaciones y coordinación de operaciones.

Esta separación te permite probar la lógica sin conectarte a MongoDB real.

9.2. Crear el servicio de productos

Con la base de datos seleccionada, obtén la colección tipada `productos` y crea el servicio de productos:

```
val products = database.getCollection<Product>("productos")
val productService = ProductService(MongoProductRepository(products))
```

9.3. Insertar documentos

Una vez que tienes el servicio, puedes insertar productos de forma sencilla. Para insertar un documento individual:

```
val product = Product(
    nombre = "Libro Kotlin",
    precio = 34.95,
    stock = 20,
```

```
        categoria = "libros",
    )

productService.register(product)

Para insertar varios documentos:

productService.registerMany(
    listOf(
        Product(nombre = "Smartphone", precio = 499.0, stock = 30, categoria = "electronica"),
        Product(nombre = "Camiseta", precio = 19.99, stock = 150, categoria = "ropa"),
        Product(nombre = "Tablet", precio = 299.0, stock = 45, categoria = "electronica"),
    )
)
```

9.4. Consultar documentos

Para obtener todos los productos:

```
val allProducts = productService.findAll()
val expensiveProducts = productService.findExpensiveProducts(50.0)
```

Internamente, `MongoProductRepository` usa filtros tipados de `KMongo`, por ejemplo:

```
collection.find(Product::precio gt minimumPrice).toList()
```

9.5. Actualizar documentos

Usando el servicio, puedes actualizar productos de varias formas. Por ejemplo, para cambiar el precio de un producto concreto:

```
productService.updatePrice("Libro Kotlin", 29.95)
productService.applyDiscountByCategory("electronica", 10)
productService.increaseStockByCategory("libros", 50)
```

También existe una operación de tipo `upsert`: actualiza si existe y crea si no existe.

En `MongoProductRepository` se implementa con la extensión de `KMongo` `replaceOneWithFilter` y `replaceUpsert()` para trabajar con el documento `Product` completo y mantener `_id` como `String`.

```
productService.upsert(
    Product(nombre = "Auriculares", precio = 89.99, stock = 60, categoria = "electronica")
)
```

9.6. Eliminar documentos

Para eliminar un producto por su nombre:

```
productService.deleteByName("Auriculares")
```

La implementación también incluye `deleteWithoutStock()` en el servicio para que puedas eliminar productos con stock menor o igual a cero.

9.7. Ejercicio 3. Catálogo de productos

9.7.1. Objetivo Practicar inserción y consulta de documentos con una colección productos.

9.7.2. Pasos

1. Usa la base de datos configurada para el taller.
2. Obtén la colección tipada productos.
3. Inserta un producto individual.
4. Inserta varios productos más.
5. Muestra todos los productos.
6. Muestra los productos con precio superior a 50 €.

9.7.3. Solución

```
val productService = ProductService(
    MongoProductRepository(database.getCollection<Product>("productos"))
)

productService.register(
    Product(nombre = "Libro Kotlin", precio = 34.95, stock = 20, categoria = "libros")
)

productService.registerMany(
    listOf(
        Product(nombre = "Smartphone", precio = 499.0, stock = 30, categoria = "electronica"),
        Product(nombre = "Camiseta", precio = 19.99, stock = 150, categoria = "ropa"),
        Product(nombre = "Tablet", precio = 299.0, stock = 45, categoria = "electronica"),
        Product(nombre = "Zapatillas", precio = 65.0, stock = 80, categoria = "ropa"),
    )
)

println(productService.findAll())
println(productService.findExpensiveProducts(50.0))
```

9.8. Ejercicio 4. Actualización de inventario

9.8.1. Objetivo Aplicar operaciones de actualización sobre productos ya insertados.

9.8.2. Pasos

1. Actualiza el precio de Libro Kotlin a 29.95 €.
2. Añade un descuento del 10 % a la categoría electronica.
3. Incrementa en 50 unidades el stock de la categoría libros.
4. Usa upsert para crear o actualizar Auriculares.
5. Elimina los descuentos de todos los documentos.

9.8.3. Solución

```
productService.updatePrice("Libro Kotlin", 29.95)
productService.applyDiscountByCategory("electronica", 10)
productService.increaseStockByCategory("libros", 50)

productService.upsert(
    Product(nombre = "Auriculares", precio = 89.99, stock = 60, categoria = "electronica")
)

productService.removeDiscounts()
```

10. Módulo 4. Proyecto integrado de biblioteca digital

10.1. Qué vas a aprender

En este módulo integrarás las operaciones anteriores en un dominio más realista: una biblioteca digital.

En este dominio usarás tres modelos:

- Autor
- Libro
- Prestamo

Y también usarás dos capas sencillas:

- LibraryRepository / MongoLibraryRepository: acceso a MongoDB.
- BibliotecaService: validaciones y operaciones de negocio.

10.2. Modelos principales

Un autor tiene un nombre, una nacionalidad y un año de nacimiento. Un libro tiene un título, el nombre del autor, un año de publicación, un género, un estado de disponibilidad y un número de copias. Un préstamo tiene el nombre del usuario, el título del libro prestado, la fecha del préstamo y un estado de devolución.

```
import org.bson.types.ObjectId

data class Autor(
    val _id: String = ObjectId().toHexString(),
    val nombre: String,
    val nacionalidad: String,
    val anioNacimiento: Int,
)

import org.bson.types.ObjectId

data class Libro(
    val _id: String = ObjectId().toHexString(),
    val titulo: String,
    val autorNombre: String,
    val anio: Int,
    val genero: String,
```

```

        val disponible: Boolean = true,
        val copias: Int,
    )

import org.bson.types.ObjectId
data class Prestamo(
    val _id: String = ObjectId().toHexString(),
    val usuario: String,
    val libroTitulo: String,
    val fechaPrestamo: String,
    val devuelto: Boolean = false,
)

```

La clase ObjectId se usa para generar un identificador único por defecto para cada documento.

10.3. Crear el servicio de biblioteca

El servicio de biblioteca se apoya en un repositorio que gestiona las tres colecciones relacionadas: autores, libros y préstamos. Para crear el servicio, primero obtén las colecciones tipadas y luego construye el servicio con el repositorio MongoDB.

```

val bibliotecaService = BibliotecaService(
    MongoLibraryRepository(
        autores = database.getCollection<Autor>("autores"),
        libros = database.getCollection<Libro>("libros"),
        prestamos = database.getCollection<Prestamo>("prestamos"),
    )
)

```

10.4. Operaciones disponibles

Las operaciones del servicio de biblioteca incluyen: - Registrar autores, libros y préstamos. - Consultar libros disponibles y préstamos pendientes. - Marcar un préstamo como devuelto y actualizar las copias del libro. - Marcar como no disponibles los libros sin copias. - Borrar los préstamos devueltos. - Obtener un resumen con el recuento de documentos.

Todas estas operaciones se implementan en el servicio BibliotecaService usando el repositorio MongoLibraryRepository para interactuar con MongoDB.

```

bibliotecaService.registerAutores(autores)
bibliotecaService.registerLibros(libros)
bibliotecaService.registerPrestamos(prestamos)

val disponibles = bibliotecaService.availableBooks()
val pendientes = bibliotecaService.pendingLoans()

bibliotecaService.returnLoan(prestamo)
bibliotecaService.updateAvailabilityWithoutCopies()
bibliotecaService.deleteReturnedLoans()

val counts = bibliotecaService.counts()

```

10.5. Ejercicio 5. Biblioteca digital completa

10.5.1. Objetivo Construir una pequeña biblioteca digital con autores, libros y préstamos.

10.5.2. Pasos

1. Crea las colecciones autores, libros y prestamos.
2. Inserta al menos 3 autores.
3. Inserta al menos 6 libros.
4. Inserta 3 préstamos.
5. Consulta los libros disponibles.
6. Consulta los préstamos no devueltos.
7. Marca un préstamo como devuelto y aumenta en 1 las copias del libro.
8. Marca como no disponibles los libros sin copias.
9. Borra los préstamos devueltos.
10. Muestra el recuento final de documentos.

10.5.3. Solución orientativa

```
val autores = listOf(
    Autor(nombre = "Miguel de Cervantes", nacionalidad = "Española", anioNacimiento = 1547),
    Autor(nombre = "Mary Shelley", nacionalidad = "Británica", anioNacimiento = 1797),
    Autor(nombre = "Frank Herbert", nacionalidad = "Estadounidense", anioNacimiento = 1920),
)

val libros = listOf(
    Libro(titulo = "El Quijote", autorNombre = "Miguel de Cervantes", anio = 1605, genero = "Ciencia ficción"),
    Libro(titulo = "Frankenstein", autorNombre = "Mary Shelley", anio = 1818, genero = "Ciencia ficción"),
    Libro(titulo = "Dune", autorNombre = "Frank Herbert", anio = 1965, genero = "Ciencia ficción"),
    Libro(titulo = "Novelas ejemplares", autorNombre = "Miguel de Cervantes", anio = 1613, genero = "Novela"),
    Libro(titulo = "El último hombre", autorNombre = "Mary Shelley", anio = 1826, genero = "Ciencia ficción"),
    Libro(titulo = "Mesías de Dune", autorNombre = "Frank Herbert", anio = 1969, genero = "Ciencia ficción"),
)

val prestamos = listOf(
    Prestamo(usuario = "Ana García", libroTitulo = "El Quijote", fechaPrestamo = "2026-05-08"),
    Prestamo(usuario = "Luis Martín", libroTitulo = "Dune", fechaPrestamo = "2026-05-08"),
    Prestamo(usuario = "María López", libroTitulo = "Frankenstein", fechaPrestamo = "2026-05-08"),
)

bibliotecaService.registerAutores(autores)
bibliotecaService.registerLibros(libros)
bibliotecaService.registerPrestamos(prestamos)

println(bibliotecaService.availableBooks())
println(bibliotecaService.pendingLoans())

bibliotecaService.returnLoan(prestamos.first())
bibliotecaService.updateAvailabilityWithoutCopies()
```

```
bibliotecaService.deleteReturnedLoans()
```

```
println(bibliotecaService.counts())
```

11. Pruebas del proyecto

11.1. Qué se prueba

Las pruebas unitarias no necesitan MongoDB real. Se centran en:

- Validación de configuración (MongoConfigTest).
- Validación de nombres de bases de datos y colecciones (DatabaseManagerTest).
- Comportamiento de ProductService con ProductRepository simulado.
- Comportamiento de BibliotecaService con LibraryRepository simulado.

11.2. Herramientas usadas

- Kotest como framework de pruebas.
- DescribeSpec como estilo de especificación.
- MockK para simular repositorios.

Ejemplo de prueba:

```
class ProductServiceTest : DescribeSpec({
    describe("ProductService") {
        it("should reject a product with negative price") {
            val repository = mockk<ProductRepository>()
            val service = ProductService(repository)

            shouldThrow<IllegalArgumentException> {
                service.register(
                    Product(nombre = "Inválido", precio = -1.0, stock = 1, categoria = "test")
                )
            }
        }
    }
})
```

12. Ejecutar el proyecto

12.1. Ejecutar las pruebas

```
./gradlew test
```

El proyecto incluye una prueba de integración en:

```
src/test/kotlin/org/iesra/tallermongo/integration/MongoWorkshopIntegrationTest.kt
```

Esta prueba usa una base de datos temporal llamada `taller_mongo_integration_test`, ejecuta operaciones reales contra MongoDB y la elimina al finalizar.

12.1.1. Comportamiento si no hay conexión configurada La prueba de integración solo se ejecuta si existe la variable `MONGODB_URI`. Si no la defines, Gradle no falla: la prueba se marcará

como omitida.

En el informe de tests verás algo similar a:

```
tests="1" skipped="1" failures="0" errors="0"
```

Esto significa que la prueba está preparada y compila, pero no se ha ejecutado contra MongoDB real porque no había una URI disponible.

12.1.2. Ejecutar la prueba de integración contra MongoDB Atlas Para ejecutarla realmente, define la URI de Atlas y lanza los tests con una JVM compatible:

```
export MONGODB_URI="mongodb+srv://usuario:password@cluster.mongodb.net/"
JAVA_HOME="/home/edu/.sdkman/candidates/java/21.0.1-tem" ./gradlew test
```

Si también quieres indicar una base de datos por defecto para la aplicación, puedes definir:

```
export MONGODB_DATABASE="taller_mongo"
```

La prueba de integración no usa esa base de datos por defecto: trabaja con `taller_mongo_integration_test` para que no mezcles datos de prueba con datos del taller.

12.2. Ejecutar la aplicación

Primero define la configuración:

```
export MONGODB_URI="mongodb+srv://usuario:password@cluster.mongodb.net/"
export MONGODB_DATABASE="taller_mongo"
```

Después ejecuta `Main.kt` desde el IDE o configura una tarea de ejecución Gradle si el proyecto la incorpora más adelante.

13. Buenas prácticas aplicadas

- No hay credenciales en el código fuente.
- Se usan variables de entorno para la conexión.
- Se usan colecciones tipadas con `data class`.
- La lógica se puede probar sin MongoDB real mediante interfaces y `MockK`.
- Las operaciones peligrosas, como eliminar bases de datos, están separadas en métodos explícitos.